

00 · Start Here — SQL & PL/SQL Interview Notes

What this is. A complete, interview-oriented set of notes on **SQL and Oracle PL/SQL**, built from a full course (video lectures, W3Schools reference pages, worked explanations and college lab work) and then checked, corrected and extended. Every chapter follows the same shape:

1. **The concept**, in plain words, with a diagram where a picture helps.
2. **Syntax and examples** — every SQL example was **run on real sample data** and the output shown is the real result.
3. **Oracle vs others** — the course database is **Oracle**, but interviews also ask how MySQL, PostgreSQL and SQL Server differ, so differences are called out.
4. **Traps** — the mistakes interviewers set on purpose.
5. **Interview corner** — the questions you will actually be asked, with short model answers.
6. **Quick recap** — a five-line summary for revision.

The roadmap

#	Chapter	What it covers	Interview weight
01	SQL Foundations	DDL/DML/DCL/TCL/DQL, data types (classic and Oracle), <code>NULL</code> and three-valued logic, operators, quoting	★★★ warm-up
02	Tables and DDL	<code>CREATE</code> , <code>CTAS</code> , <code>ALTER</code> , <code>DROP</code> vs <code>TRUNCATE</code> vs <code>DELETE</code> , identity / sequences / <code>AUTO_INCREMENT</code>	★★★
03	Constraints and Keys	<code>NOT NULL</code> , <code>UNIQUE</code> , <code>PRIMARY KEY</code> , <code>FOREIGN KEY (ON DELETE)</code> , <code>CHECK</code> , <code>DEFAULT</code> , naming, error codes, kinds of keys	★★★
04	SELECT — Querying and Filtering	<code>WHERE</code> , <code>AND/OR</code> precedence, <code>IN</code> , <code>BETWEEN</code> , <code>LIKE</code> , <code>ORDER BY</code> , <code>CASE</code> , top-N, aliases, order of execution	★★★★★
05	DML — INSERT, UPDATE, DELETE	all <code>INSERT</code> forms, <code>INSERT ALL</code> , <code>INSERT ... SELECT</code> , subquery updates, <code>COMMIT</code> / <code>ROLLBACK</code> / <code>SAVEPOINT</code>	★★★
06	Aggregation and GROUP BY	aggregates, <code>GROUP BY</code> , <code>WHERE</code> vs <code>HAVING</code> , duplicates, <code>ROLLUP</code> / <code>CUBE</code>	★★★★★
07	Joins and Set Operators	inner / outer / cross / self joins, <code>ON</code> vs <code>WHERE</code> , anti-joins, <code>UNION</code> family	★★★★★
08	Subqueries	scalar / row / column / table, correlated, <code>EXISTS</code> , <code>IN</code> vs <code>NOT IN</code> , <code>ANY</code> / <code>ALL</code>	★★★★★
09	Built-in Functions	string, numeric, date, conversion, <code>NULL</code> , regular-expression functions	★★★★
10	Views and Indexes	views, <code>WITH CHECK OPTION</code> , B-tree indexes, composite order, when indexes don't help	★★★★
11	Window Functions	<code>OVER</code> , <code>PARTITION BY</code> , <code>RANK</code> / <code>DENSE_RANK</code> / <code>ROW_NUMBER</code> , running totals, top-N per group	★★★★★
12	PL/SQL Fundamentals	blocks, variables and scope, <code>%TYPE</code> , <code>IF</code> / <code>CASE</code> , loops, strings, <code>VARRAY</code>	★★★★
13	Procedures, Functions and Packages	parameter modes, calling conventions, recursion, package spec / body	★★★★

#	Chapter	What it covers	Interview weight
14	Cursors and Records	implicit / explicit cursors, attributes, <code>FOR</code> loops, <code>%ROWTYPE</code> , records	★★★★
15	Triggers	timing, row vs statement, <code>:OLD</code> / <code>:NEW</code> , firing order, <code>INSTEAD OF</code> , compound triggers, mutating table	★★★★
16	Practice Problems	34 executed SQL problems + 10 PL/SQL exercises	practice
17	Quick Revision Cheat Sheet	the whole set on a few pages	last-minute

Extras — a second folder of deeper topics

Six additional chapters live in the **Extras** folder next to these files. They go beyond the course material and cover what interviews ask that the lectures didn't.

#	Extra chapter	What it covers	Interview weight
E1	CTEs and Recursive Queries	<code>WITH</code> , recursive CTEs, Oracle <code>CONNECT BY</code> , hierarchies, number/date generators	★★★★★
E2	Transactions, ACID and Isolation	ACID, dirty / non-repeatable / phantom reads, isolation levels, locks, <code>FOR UPDATE</code> , deadlocks	★★★★★
E3	Normalization and Schema Design	anomalies, functional dependencies, 1NF → BCNF, ER modelling, keys, star schema	★★★★★
E4	Query Tuning and Execution Plans	optimizer and statistics, reading plans, access paths, join methods, SARGable predicates, rewrites	★★★★
E5	Advanced SQL Interview Problems	28 executed problems: gaps and islands, running totals, pivot, division, duplicates, median	★★★★★
E6	PL/SQL Exception Handling	predefined / user-defined exceptions, <code>RAISE_APPLICATION_ERROR</code> , <code>EXCEPTION_INIT</code> , propagation, logging	★★★★

Suggested paths

Time available	Read
One evening	04 → 06 → 07 → 08 → 11, then the "Interview corner" of every other chapter, then 17
One week	01–03 (day 1) · 04–05 (day 2) · 06–07 (day 3) · 08–11 (day 4) · 12–13 (day 5) · 14–15 (day 6) · 16 + 17 (day 7)
Oracle / PL/SQL role	everything, and do Part G of chapter 16 by hand

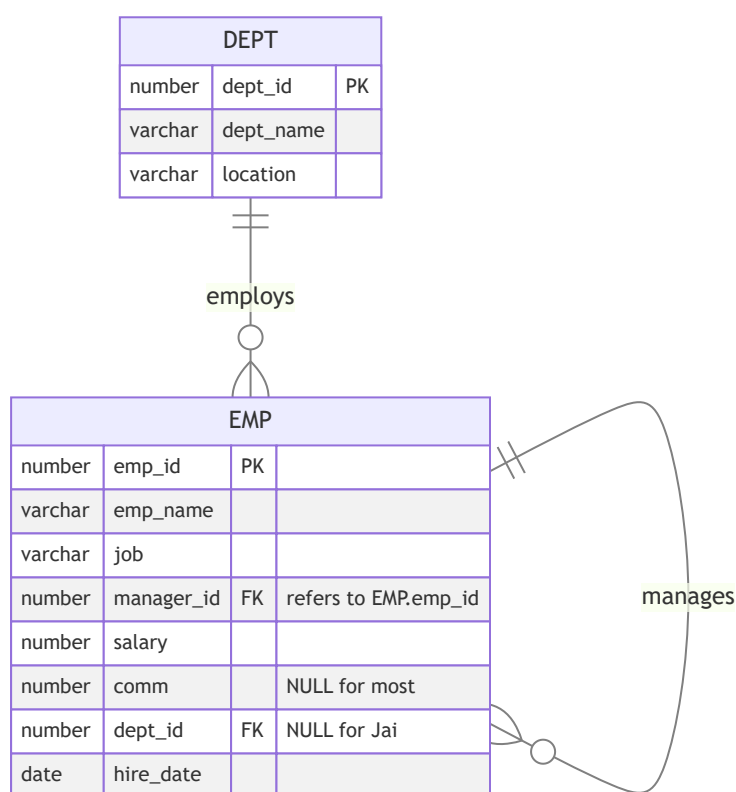
How to read the callouts

Marker	Meaning
Oracle vs others.	a place where Oracle differs from MySQL / PostgreSQL / SQL Server

Marker	Meaning
Trap.	a classic mistake or a favourite interviewer puzzle
Correction.	a common tutorial claim that is actually wrong, and the right version
Q. ...	an interview question, with the model answer beneath

The sample database

Most examples use two small tables — a company with **departments** and **employees**. They are built so that the interesting cases exist: an employee with **no department** (Jai), a department with **no employees** (HR), employees who **tie** on salary, a **manager hierarchy**, and a mix of **NULL** and non-NULL commissions.



dept

dept_id	dept_name	location
10	Management	Chennai
20	Engineering	Bengaluru
30	Sales	Mumbai
40	HR	Pune

emp

emp_id	emp_name	job	manager_id	salary	comm	dept_id	hire_date
101	Asha	CEO	NULL	90000	NULL	10	2015-01-10

emp_id	emp_name	job	manager_id	salary	comm	dept_id	hire_date
102	Bala	Manager	101	70000	NULL	20	2016-03-15
103	Chitra	Developer	102	50000	NULL	20	2018-07-01
104	Dinesh	Developer	102	50000	NULL	20	2019-02-20
105	Esha	Tester	102	40000	NULL	20	2020-11-05
106	Farhan	Manager	101	65000	NULL	30	2017-05-30
107	Gita	Sales Rep	106	35000	5000	30	2021-01-12
108	Hari	Sales Rep	106	35000	3000	30	2022-06-18
109	Isha	Intern	103	20000	NULL	20	2024-01-08
110	Jai	Consultant	101	45000	NULL	NULL	2023-04-01

Dates and "today". Examples show dates in ISO form (2015-01-10) for readability; an Oracle session prints them as 10-JAN-15 (its NLS_DATE_FORMAT). Wherever a function needs the current date, **"today" is 16-AUG-2025**, so SYSDATE gives that date and every result is reproducible.

Build it yourself — paste this into SQL Developer (run as a script with F5):

```

CREATE TABLE dept (
    dept_id    NUMBER          PRIMARY KEY,
    dept_name  VARCHAR2(30),
    location   VARCHAR2(30)
);

INSERT INTO dept VALUES (10, 'Management', 'Chennai');
INSERT INTO dept VALUES (20, 'Engineering', 'Bengaluru');
INSERT INTO dept VALUES (30, 'Sales', 'Mumbai');
INSERT INTO dept VALUES (40, 'HR', 'Pune');

CREATE TABLE emp (
    emp_id      NUMBER          PRIMARY KEY,
    emp_name    VARCHAR2(30),
    job         VARCHAR2(30),
    manager_id  NUMBER,
    salary      NUMBER,
    comm        NUMBER,
    dept_id     NUMBER,
    hire_date   DATE
);

INSERT INTO emp VALUES (101, 'Asha', 'CEO', NULL, 90000, NULL, 10, DATE '2015-01-10');
INSERT INTO emp VALUES (102, 'Bala', 'Manager', 101, 70000, NULL, 20, DATE '2016-03-15');
INSERT INTO emp VALUES (103, 'Chitra', 'Developer', 102, 50000, NULL, 20, DATE '2018-07-01');
INSERT INTO emp VALUES (104, 'Dinesh', 'Developer', 102, 50000, NULL, 20, DATE '2019-02-20');
INSERT INTO emp VALUES (105, 'Esha', 'Tester', 102, 40000, NULL, 20, DATE '2020-11-05');
INSERT INTO emp VALUES (106, 'Farhan', 'Manager', 101, 65000, NULL, 30, DATE '2017-05-30');
INSERT INTO emp VALUES (107, 'Gita', 'Sales Rep', 106, 35000, 5000, 30, DATE '2021-01-12');
INSERT INTO emp VALUES (108, 'Hari', 'Sales Rep', 106, 35000, 3000, 30, DATE '2022-06-18');
INSERT INTO emp VALUES (109, 'Isha', 'Intern', 103, 20000, NULL, 20, DATE '2024-01-08');
INSERT INTO emp VALUES (110, 'Jai', 'Consultant', 101, 45000, NULL, NULL, DATE '2023-04-01');

COMMIT;

```

Some chapters use small extra tables that appear in the course material (`customers` / `orders` , `products` , `sales` , `students` ...); each is shown where it is used.

Running SQL and PL/SQL

Task	How (Oracle SQL Developer)
Run one statement	put the cursor in it, <code>Ctrl + Enter</code> → result grid
Run a whole script	<code>F5</code> → <i>Script Output</i>
See <code>DBMS_OUTPUT</code> text	<code>SET SERVEROUTPUT ON</code> ; first, then run the block
Run a PL/SQL block	finish it with <code>END</code> ; and a line containing only <code>/</code>

Task	How (Oracle SQL Developer)
Save your work	run <code>COMMIT;</code> — auto-commit is off; DDL commits by itself
<code>ORA-12505</code> on connect	choose the <i>Service name</i> option instead of <i>SID</i> (Chapter 01)

The dialect map — Oracle at a glance vs the others

The single most useful table when an interviewer says "and in MySQL?".

Task	Oracle	MySQL	PostgreSQL	SQL Server
Top <i>n</i> rows	<code>FETCH FIRST n ROWS ONLY</code> (12c+) · <code>ROWNUM</code> <code><= n</code>	<code>LIMIT n</code>	<code>LIMIT n</code>	<code>SELECT TOP n</code> · <code>OFFSET ... FETCH</code>
Pagination	<code>OFFSET m ROWS FETCH NEXT n ROWS ONLY</code>	<code>LIMIT n OFFSET m</code>	<code>LIMIT n OFFSET m</code>	<code>OFFSET m ROWS FETCH NEXT n ROWS ONLY</code>
String concatenation	<code> </code> · <code>CONCAT(a,b)</code>	<code>CONCAT(a,b,...)</code>	<code> </code>	<code>+</code> · <code>CONCAT</code>
<code>'abc'</code> · <code> </code> <code>NULL</code>	<code>'abc'</code> (NULL ≈ empty)	NULL	NULL	NULL (+)
Empty string <code>''</code>	is NULL	a real value	a real value	a real value
<code>SELECT</code> without a table	<code>FROM dual</code>	allowed	allowed	allowed
Current date/time	<code>SYSDATE</code> , <code>SYSTIMESTAMP</code>	<code>NOW()</code> , <code>CURDATE()</code>	<code>NOW()</code> , <code>CURRENT_DATE</code>	<code>GETDATE()</code> , <code>SYSDATETIME()</code>
NULL replacement	<code>NVL(a,b)</code> , <code>COALESCE</code>	<code>IFNULL(a,b)</code> , <code>COALESCE</code>	<code>COALESCE</code>	<code>ISNULL(a,b)</code> , <code>COALESCE</code>
Auto-generated key	<code>GENERATED ... AS IDENTITY</code> (12c+) · <code>SEQUENCE</code>	<code>AUTO_INCREMENT</code>	<code>SERIAL</code> · <code>GENERATED ... AS IDENTITY</code>	<code>IDENTITY(1,1)</code>
Text type	<code>VARCHAR2(n)</code> (max 4000) · <code>CLOB</code>	<code>VARCHAR(n)</code> · <code>TEXT</code>	<code>VARCHAR(n)</code> · <code>TEXT</code>	<code>VARCHAR(n)</code> · <code>NVARCHAR(MAX)</code>
Boolean column	none before 23ai (<code>NUMBER(1)</code>)	<code>BOOLEAN (= TINYINT(1))</code>	<code>BOOLEAN</code>	<code>BIT</code>
Date-only type	<code>DATE</code> (includes time!)	<code>DATE</code>	<code>DATE</code>	<code>DATE</code>
Change a column type	<code>MODIFY col type</code>	<code>MODIFY COLUMN col type</code>	<code>ALTER COLUMN col TYPE type</code>	<code>ALTER COLUMN col type</code>
Table alias	<code>FROM emp e</code> — no AS	<code>AS</code> optional	<code>AS</code> optional	<code>AS</code> optional
Anti- MINUS	<code>MINUS</code>	<code>EXCEPT</code> (8.0.31+)	<code>EXCEPT</code>	<code>EXCEPT</code>
<code>FULL OUTER JOIN</code>	yes	✗ (emulate with <code>UNION</code>)	yes	yes
<code>5 / 2</code>	<code>2.5</code>	<code>2.5000</code>	<code>2</code> (integer division)	<code>2</code> (integer division)

Task	Oracle	MySQL	PostgreSQL	SQL Server
Modulo	MOD(a, b)	% · MOD	% · MOD	%
Alias in GROUP BY	✗ (through 21c)	✓	✓	✗
LIKE bracket wildcards []	✗ (use REGEXP_LIKE)	✗	✗	✓
DDL in a transaction	auto-commits	auto-commits	transactional	transactional
TRUNCATE rollback	✗	✗	✓	✓
Procedural language	PL/SQL	stored routines (SQL/PSM)	PL/pgSQL	T-SQL
String literal quoting	'...' (' ' for a quote)	'...'	'...'	'...'
Identifier quoting	"..."	`...`	"..."	[...]
Case-sensitive string compare	yes	no (default collation)	yes	no (default collation)
Window functions	8i+	8.0+	8.4+	2005+

Rule of thumb for answers. State the standard idea first ("limit the rows returned"), then name the Oracle form, then mention the others: "In Oracle 12c and later it's `FETCH FIRST n ROWS ONLY`; before that you'd sort in a subquery and filter on `ROWNUM`; MySQL and PostgreSQL use `LIMIT`, SQL Server `TOP`."

A note on the sources

The notes were assembled from the course material and then **verified line by line**. Where a source contained a mistake, the correct version is given and marked **Correction** — for example the `!= ANY` vs `NOT IN` confusion, the row-subquery example that cannot run, the claim that `[ ]` wildcards fail only in MySQL and PostgreSQL, and named `DEFAULT` constraints. Read those boxes; they are exactly the points where a confident-but-wrong answer costs marks.